Reflection

CS330

Elijah Bastien

6/20/26

The visual arrangement of this 3D scene were chosen to build a narrative around an aspring sports artist working late into the night. To translate this concept into a 3D workspace, everyday objects were deconstructed into their foundational geometric primitives. The core of the scene features a dark navy blue notebook, constructed using a flattened BoxMesh (scaleXYZ = [2.5f, 0.3f, 3.5f]), paired with a slender, elongated CylinderMesh positioned at an angle to represent a drawing pen. To portray the "sports artist" theme, a SphereMesh textured with a gray basketball skin was introduced as a piece of inspiration on the artists desk. The late night artistic aesthetic is tied together by a textured ceramic coffee mug, built by intersecting a vertical CylinderMesh for the body and a 90 degree rotated TorusMesh for the handle. Implementing these objects required precise transformations and coordinate spacing to prevent mesh clipping and ensure structural realism. Using a common plane as the desk surface (Y = 0.0f), every object’s bounding box had to be carefully calculated. For example, because the notebook stack possesses a height scale of 0.3f, its center position was offset to Y = 0.15f to sit perfectly flush with the table. The pen’s coordinates were elevated to Y = 0.38f so it would rest naturally on top of the notebook without clipping through the geometry. Also a repeating coordinate scale was applied to the wood grain desk texture to prevent unnatural stretching, while specular and

shininess properties were configured across meshes to differentiate glossy plastic, matte paper, and coarse rubber material shaders.

An interactive camera system was also implemented that allows users to traverse and explore the 3D workspace across the global X, Y, and Z axes. The camera movement relies on a dual input configuration combining keyboard layout controls with precision mouse adjustments. The W and S keys dictate forward and backward translation along the camera's target vector, while the A and D keys control lateral movements. To navigate height variations across the desk environment, the Q and E keys are bound to vertical camera movement along the positive and negative Y axes. Changes to the camera's orientation without altering its physical position are managed via the mouse cursor. By calculating the mouse movement offsets from frame to frame, the system updates the camera’s pitch and yaw. In terms of speed modulation, mouse scroll events are registered to dynamically scale the camera's movement speed modifier. This allows the user to accelerate across broad expanses of the wood panel table or slow down to inspect the fine specular glints on the pen and sphere meshes.

Beyond detailed documentation and comments this application relies on custom, highly reusable object oriented functions declared within the SceneManager class. Rather than writing redundant OpenGL pipeline calls for each primitive shape, specific tasks are isolated into modular helper functions. CreateGLTexture() abstracts the complex image parsing handled by the stb_image library. It automatically handles channel checks (RGB vs. RGBA), configures wrapping parameters (GL_REPEAT), generates necessary mipmaps, maps the texture to memory, and binds it to a unique string tag. This allows a developer to load an indefinite number of image files using a single line of code. SetTransformations() streamlines the matrix multiplication pipeline. It accepts raw vectors for scale and translation alongside Euler angles for

rotation, internally computes the necessary transformation matrices via GLM math libraries, and automatically uploads the combined modelView matrix to the active shader program. Lastly SetShaderMaterial() and SetShaderColor() allow on the fly variations of surface textures, ambient light reflection parameters, and shininess values. This architecture ensures that the rendering loop remains clean and scalable, as drawing any asset simply requires setting its state parameters before calling the primitive's native draw command.